

Agentic AI 따라해보기

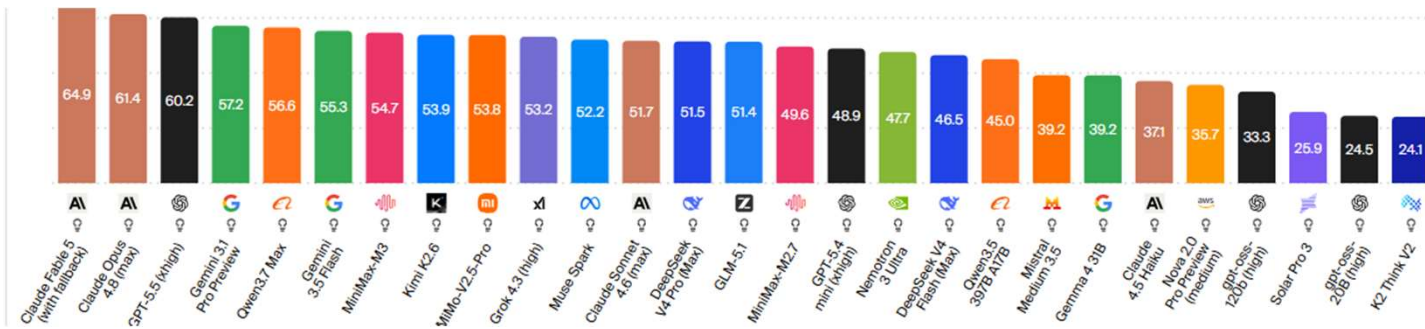
20220300 김종현

2026-06-10

목차

1. Agentic AI란?
2. 주제를 선택한 이유
3. 구현

Agentic AI란?



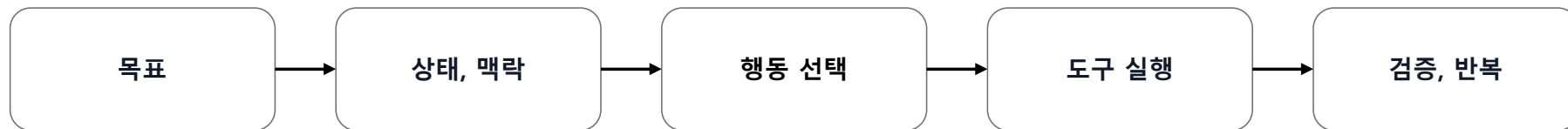
성능

이미 작년 기준으로도 사용에는 문제가 없었다

성능 말고 다른 개선?

성능이 아닌 구조적인 개선을 하면 어떨까?

Agentic AI란?



기존 채팅 방식과 차이

단순 답변 생성에서 끝나는 것이 아닌 파일 읽기, 실행, 수정, 검증까지 영향을 준다.

핵심

모델 자체 성능보다 반복, 관리, 검증과 같은 구조가 중요

Agentic AI란?

chessboardwidget.h
파일

chessTestReal.pro
파일
game.cpp

game.cpp
파일

game.h
파일

images.qrc
파일

main.cpp
파일

mainwindow.h
파일

mainwindow.ui
파일

이 파일들 분석해줘

이 프로젝트 분석해줘

17s 동안 작업 중입니다

프로젝트 구조부터 훑겠습니다. 무엇으로 만들어졌는지, 진입점과 핵심 모듈이 어디인지 먼저 잡고 요약하겠습니다.

Get-ChildItem -Force | Select-Object Name,Mode,Length; Write-Host '---'; if (Get-Command rg -ErrorAction Sil...

이제 엔트리포인트와 클래스 경계를 봅니다. Qt 위젯 기반 체스 앱으로 보이니, UI, 게임 로직, AI, 저장 로직이 어떻게 나뉘는지 확인하겠습니다.

실행 중인 Get-Content main.cpp

Agentic AI란?



Codex

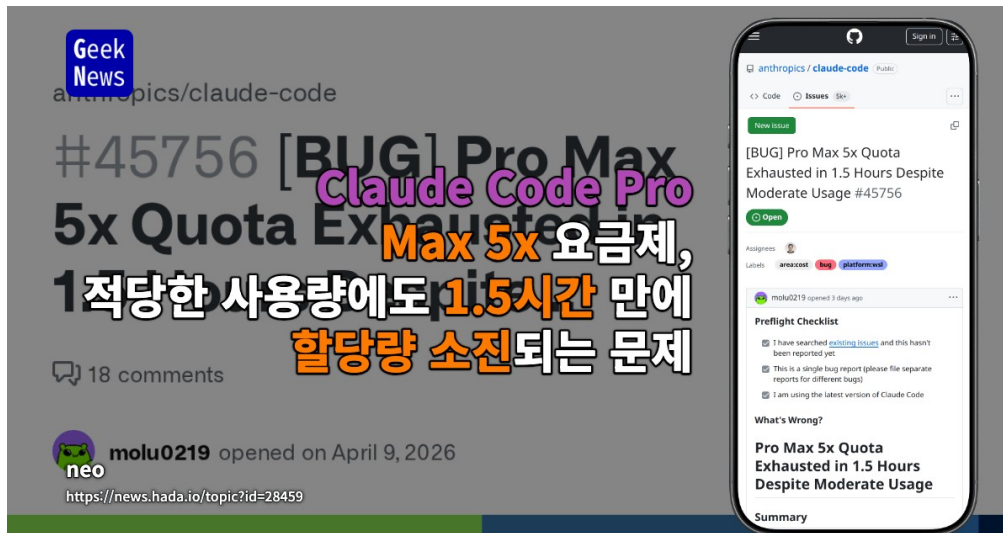
OpenAI에서 개발된 모델



Claude

Anthropic에서 개발된 모델

Agentic AI의 문제점?



비용 문제

기존 채팅뿐만 아니라 다양한 권리를 가져 토큰 소모량 증가

보안적인 문제

사용되는 내용들이 모두 서버에 전송되는 문제도 존재

오늘의 주제 – Agentic AI를 따라해보자

1. 비용 문제

로컬 LLM을 활용한다면 하드웨어자원만 있으면 되니 비용에서 자유롭다.

2. 보안 문제

로컬에서 내용들이 처리되기에 민감한 정보에 대한 보안적인 부분도 자유롭다.

3. 클라우드 생태계

클라우드 연결이 필요 없어 인터넷이 사용할 수 없거나 폐쇄적인 곳에서도 활용 가능하다.

-> 어떻게 구조를 잘 구현한다면 로컬 LLM으로도 성능이 나오지 않을까?

구현

agent.py

agent

프롬프트 구성, LLM 호출, 반복

actions.py

action

모델이 선택할 수 있는 행동의 종류와 입력 형식을 정의

executor.py

executor

파일 읽기, 검색, 명령 실행, 차이점 확인

workspace.py

workspace

Workspace 경로 유지, 저장소 관리

cli.py

cli

환경변수 관리, 결과 출력

agent.py

```
local_codex_cli > agent.py > ...  
  
SYSTEM_PROMPT = """You are a local coding agent that must operate only inside the provided workspace.  
Always respond with exactly one JSON object and no surrounding markdown.  
Never emit free-form text.  
Allowed actions:  
- list_files  
- read_file  
- search_files  
- run_command  
- apply_patch  
- git_diff  
- final_answer
```

프롬프트

모델이 정해진 action 중 하나를 JSON으로 보내도록 강제

```
def run(self, task: str) -> AgentResult:  
    agent_md = self._load_agents_md()  
    snapshot = build_workspace_snapshot(self.workspace_root)  
    self.messages = [  
        {"role": "system", "content": SYSTEM_PROMPT},  
        {  
            "role": "system",  
            "content": (  
                f"Workspace root: {self.workspace_root}\n"  
                "Repository snapshot:\n"  
                f"{snapshot}\n"  
                "If AGENTS.md content is present, follow it as an additional instruction.\n"  
                f"AGENTS.md:\n{agent_md or '(not found)}"  
            ),  
        },  
        {"role": "user", "content": task},  
    ]
```

LLM에게 전달

LLM에게 규칙들이나 작업 폴더, 구조와 같은 내용을 전달

agent.py

```
for iteration in range(1, self.config.max_iterations + 1):
    self.console.log(f"LLM turn {iteration}: requesting next action")
    action = self._next_action()
    self.console.log(f"LLM turn {iteration}: action={action.action}")
    self.messages.append({"role": "assistant", "content": action.model_dump_json()})

    if isinstance(action, FinalAnswerAction):
        if not context.verification_commands:
            auto_commands = discover_verification_commands(self.workspace_root)
            if auto_commands:
                self.console.log("Running automatic verification before final answer")
                for command in auto_commands:
                    result = executor.execute(
                        RunCommandAction(
                            command=command,
                            reason="Automatic verification before final answer",
                        )
                    )
                    self.messages.append({"role": "tool", "content": json.dumps(result, ensure_ascii=False)})
                self.messages.append(
                    {
                        "role": "system",
                        "content": (
                            "Verification commands were executed automatically. "
                            "Now return an updated final_answer JSON that reflects those results."
                        ),
                    },
                )
                continue
            return AgentResult(
                summary=action.summary,
                changed_files=sorted(set(action.changed_files) | context.changed_files),
                verification=action.verification or context.verification_commands,
                risks=action.risks,
                iterations=iteration,
            )

    result = executor.execute(action)
    self.console.print(Panel.fit(result["summary"], title=f"Action Result: {action.action}"))
    self.messages.append({"role": "tool", "content": json.dumps(result, ensure_ascii=False)})

raise RuntimeError(f"Maximum iterations exceeded: {self.config.max_iterations}")
```

반복 구조

LLM이 action을 선택, executor가 실행, 그 결과를 LLM에게 전달

최종 답안이 나올 때까지 반복

agent.py, actions.py

```
def next_action(self):
    payload = {
        "model": self.config.model,
        "messages": self.messages,
        "temperature": 0.2,
        "response_format": {"type": "json_object"},
    }
    headers = {"Authorization": f"Bearer {self.config.api_key}" if self.config.api_key else {}}
    with httpx.Client(base_url=self.config.base_url, timeout=60.0) as client:
        for attempt in range(1, 4):
            response = client.post("/chat/completions", headers=headers, json=payload)
```

로컬 LLM 호출

API를 통해 요청을 전송

```
ActionModel = Annotated[
    Union[
        ListFilesAction,
        ReadFileAction,
        SearchFilesAction,
        RunCommandAction,
        ApplyPatchAction,
        GitDiffAction,
        FinalAnswerAction,
    ],
    Field(discriminator="action"),
]
```

```
class FinalAnswerAction(BaseModel):
    action: Literal["final_answer"] = "final_answer"
    summary: str
    changed_files: list[str]
    verification: list[str]
    risks: list[str]
```

Action 검증

모델 응답이 json이라 하더라도 action에서 요구하는 구조인지 확인

executor.py, workspace.py

```
def execute(self, action) -> dict[str, Any]:
    if isinstance(action, ListFilesAction):
        return self._list_files(action)
    if isinstance(action, ReadFileAction):
        return self._read_file(action)
    if isinstance(action, SearchFilesAction):
        return self._search_files(action)
    if isinstance(action, RunCommandAction):
        return self._run_command(action)
    if isinstance(action, ApplyPatchAction):
        return self._apply_patch(action)
    if isinstance(action, GitDiffAction):
        return self._git_diff(action)
    raise TypeError(f"Unsupported action: {action}")
```

```
def ensure_workspace_path(workspace_root: Path, relative_path: str) -> Path:
    candidate = (workspace_root / relative_path).resolve()
    root = workspace_root.resolve()
    if candidate != root and root not in candidate.parents:
        raise ValueError(f"Path escapes workspace: {relative_path}")
    return candidate
```

무슨 행동을 할지 선택

모델에서 직접 처리하는 것이 아니라 파일 읽기, 명령 실행과 같은 부분은 executor가 대신 행동

Workspace 고정

해당 프로그램이 다른 파일을 수정하지 않도록 막는 부분

테스트

```
def test_calculate_total_with_tax(self):
    items = [
        {"name": "keyboard", "price": 10000, "quantity": 2},
        {"name": "mouse", "price": 5000, "quantity": 1},
    ]
    self.assertEqual(calculate_total(items, tax_rate=0.1), 27500)

def test_apply_discount(self):
    self.assertEqual(apply_discount(10000, 20), 8000)
```

```
cart.py > apply_discount
1 def calculate_total(items, tax_rate=0.1):
2     total = 0
3     for item in items:
4         total += item["price"] * item["quantity"]
5     return total
6
7
8 def apply_discount(total, discount_percent):
9     return total - (total * discount_percent)
```

틀린 코드 준비

세금 고려 X
할인 계산도 이상하다

수정할 부분

세금 비율만큼 비용 추가
할인하는 공식 수정

테스트

```
$env:LOCAL_LLM_BASE_URL = "http://localhost:1234/v1"
$env:LOCAL_LLM_API_KEY = "lm-studio"
$env:LOCAL_LLM_MODEL = "openai/gpt-oss-20b"
```

```
[01:40:39] Running command: python -m unittest -v
LLM turn 1: requesting next action
[01:40:42] LLM turn 1: action=list_files
List_files: Listed 2 paths
LLM turn 2: requesting next action
[01:40:45] LLM turn 2: action=run_command
Running command: run_command
run_command: Command exited with code 1
[01:40:46] LLM turn 3: requesting next action
[01:40:49] LLM turn 3: action=final_answer
Running automatic verification before final answer
Running command: python -m unittest -v
LLM turn 4: requesting next action
[01:40:51] LLM turn 4: action=final_answer
Final summary:
Summary: Tests failed due to incorrect total calculation and discount logic.
Changed files: -
Verification: python -m unittest -v
Risks: -
Iterations: 4
PS C:\ForStudy\localcodex-agent-demo> |
```

```
{
  "messages": [
    {
      "role": "assistant",
      "content": "({\"summary\":\"\\nInitial test failures due to incorrect calculations.\\n\\n - calculate_total ignored tax_rate, returning subtotal only.\\n\\n - fixes: add tax calculation and divide discount by 100.\\n\\n\", \"changed_files\": [], \"verification\": [], \"risks\": []}),",
      "reasoning": "",
      "tool_calls": []
    },
    {
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 1395,
    "completion_tokens": 66,
    "total_tokens": 1461,
    "completion_tokens_details": {
      "reasoning_tokens": 0
    }
  }
}
```

GPT-OSS 활용

오래된 모델이지만 간단하게 활용 가능하다

결과

총합 결과와 할인이 이상하게 되어있다 알림

테스트

```
read_file next:
LLM turn 6: requesting next action
[16:01:58] LLM turn 6: action=read_file
read_file: Read piece.cpp
LLM turn 7: requesting next action
[16:02:04] LLM turn 7: action=final_answer
LLM turn 8: requesting next action
[16:02:11] LLM turn 8: action=final_answer
Final summary:
Summary: Analyzed piece.cpp, piece.h. The files define chess piece abstractions and implement piece-specific move
generation logic, with Game supplying board state and special-rule context.
Changed files: -
Verification: -
Risks: -
Suggested changes: -
Files read: piece.cpp, piece.h
File roles: piece.cpp: implements runtime logic and movement rules. | piece.h: declares interfaces, types, and
piece-related APIs.
Key structures: class Game: appears in piece.h. | class Piece: appears in piece.h. | class Pawn: appears in piece.h. |
class Knight: appears in piece.h. | class Bishop: appears in piece.h. | class Rook: appears in piece.h. | enum
PieceColor: appears in piece.h. | enum PieceType: appears in piece.h.
Logic flow: piece.cpp: getPossibleMove is a core entry point for generating legal move candidates. | piece.cpp:
addStraightMove is a shared helper for sliding pieces such as bishop, rook, and queen. | piece.cpp: movement generation
depends on Game state for castling or en passant. | piece.h: getPossibleMove is a core entry point for generating legal
move candidates.
Open questions: piece.cpp: depends on Game, so complete move legality still requires checking Game-side validation. |
piece.h: depends on Game, so complete move legality still requires checking Game-side validation.
Iterations: 3
```

모델의 한계

```
Read piece.n
LLM turn 3: requesting next action
Rejected action: final_answer is for
read_file next.
LLM turn 4: requesting next action
Rejected action: final_answer is for
read_file next.
LLM turn 5: requesting next action
Rejected action: final_answer is for
read_file next.
LLM turn 6: requesting next action
```

계속해서 벗어나는 모델

A를 원해서 강하게 명령해서 시키면 먼저 끝내려는 경향이 있기에 프로그램에서 벗어나지 않도록 강제해야 함

모델의 출력

방향은 강제한다고 해도 그 출력물이 원하는 대로 나오지 않는다 단순히 읽었다고만 하는 경우도 자주 발생

결론

- 이번 구현을 통해 로컬 LLM도 파일 읽기, 코드 분석, 수정, 실행 결과 확인을 반복하는 Agentic AI 구조를 어느 정도 따라할 수 있음을 확인했습니다.
- 비슷하게 구현은 했지만 많이 부족합니다
- JSON으로 주고 받는 방식이 문제인지 다른 모델에서는 오류가 발생하는 문제도 존재
- 이러한 도구의 성능을 보완하는 것과 모델 호환성에 대해서도 개선할 예정